

Содержание

1. Введение	3
1.1. Архитектура моддинг-системы	3
1.2. Жизненный цикл загрузки модов	3
2. Быстрый старт: создание первого мода	4
2.1. Структура папки мода	4
2.2. Файл mod.json — манифест мода	5
2.3. Добавление предметов	5
2.4. Установка и активация мода	5
3. Справочник манифеста mod.json	5
3.1. Поля манифеста	5
3.2. Поле data — типы данных	6
3.3. Total Conversion — полная конверсия	7
3.4. Валидация манифеста	7
4. Декларативный моддинг: JSON-данные	7
4.1. Предметы (items)	7
Операторы слияния mergeDeep	8
4.2. Биомы (biomes)	8
4.3. Монстры (monsters)	8
4.4. Бедствия (disasters)	9
4.5. Расы (races)	10
4.6. Профессии (professions)	10
4.7. Черты характера (traits)	10
4.8. Конфигурация мира (world_config)	10
5. Скриптовый моддинг: ModAPI v3.0	11
5.1. Система хуков (on/emit)	11
5.2. Hook Chain System (hookFunction)	11
5.3. Инъекции в промпт ИИ	12
5.4. Кастомные ГМ-команды	12
5.5. Фильтры (prompt, response, text)	12
5.6. Система виджетов (registerWidget)	12
5.7. CSS-инъекции (addStyle)	13
5.8. Горячие клавиши (registerHotkey)	13
5.9. Локализация (addTranslations/setString)	13
5.10. Система сохранений (registerSaveData)	13
5.11. Взаимодействие с C++ ядром	13
5.12. Чтение файлов мода	14
5.13. Виртуальная файловая система (resolveAsset)	14

5.14. Уведомления и вкладки настроек	14
5.15. Выгрузка мода (unloadMod)	14
6. Тотальная конверсия (Total Conversion)	14
6.1. Структура мода тотальной конверсии	15
6.2. Ключевые техники тотальной конверсии	15
7. Нативный C/C++ моддинг (Metera Mod SDK)	15
7.1. Жизненный цикл нативного плагина	16
7.2. Обязательные экспортируемые функции	16
7.3. Таблица API (MeteraAPI)	16
7.3.1. Запросы мира (read-only)	16
7.3.2. Мутации мира (deferred)	17
7.4. Система callback-функций	18
7.5. Пример нативного плагина	19
7.6. Безопасность нативных плагинов	19
8. Система безопасности: Песочница (Sandbox)	19
8.1. Принцип работы	20
8.2. Проксирование window и document	20
8.3. Публичный фасад ModAPI	21
8.4. Отключение песочницы	21
9. Meterea Creation Kit (ModKit.py)	21
9.1. Возможности Creation Kit	21
9.2. Запуск Creation Kit	22
10. Менеджер модификаций (UI)	22
10.1. Интерфейс менеджера	22
10.2. Безопасный режим	22
11. Примеры модов из репозитория	22
11.1. cyberpunk_core — Neon & Chrome: Total Conversion	23
11.2. neon_abyss — Neon Abyss	23
Приложение А: Полный справочник ModAPI	23
Приложение В: Коды ошибок MeteraResult	24

1. Введение

Chronicles of Meterea (Метера) — это текстовая ролевая игра с симуляцией мира, работающая на платформе Electron. Ядро симуляции мира написано на C++ (Nexus Engine), а интерфейс и игровая логика реализованы на JavaScript. Игра поддерживает три уровня модификации: кэжуальный (декларативные JSON-данные), продвинутый (JavaScript API ModKit v3.0) и экспертный (нативные C/C++ плагины через Metera Mod SDK).

Система моддинга построена по принципу слоистой архитектуры: моды загружаются последовательно в порядке приоритета, каждый мод может расширять или переопределять данные предыдущих. Это позволяет создавать как небольшие дополнения (новые предметы, расы, биомы), так и тотальные конверсии, полностью заменяющие сеттинг игры. Менеджер модов с графическим интерфейсом обеспечивает удобное управление порядком загрузки, проверку зависимостей и безопасный режим для блокировки потенциально опасных нативных плагинов.

Meterea Creation Kit (ModKit.py) — это отдельное настольное приложение на PyQt6, предоставляющее визуальный редактор для создания и редактирования модов без необходимости ручного редактирования JSON-файлов. Оно включает текстовый редактор с подсветкой синтаксиса, визуальный древовидный редактор JSON, валидатор и функцию упаковки мода в ZIP-архив для распространения.

1.1. Архитектура моддинг-системы

Архитектура состоит из следующих ключевых компонентов, каждый из которых играет определенную роль в конвейере загрузки и выполнения модификаций:

Компонент	Технология	Назначение
ModLoader.js	JavaScript	Загрузка манифестов модов, выполнение скриптов, глубокое слияние JSON-данных
ModAPI (window.Mod API)	JavaScript	Публичный API для модов: хуки, команды, фильтры, виджеты, стили
ModLoaderIntegration.js	JavaScript	Мост между JS-модами и C++ ядром через IPC (Electron)
ModManagerUI.js	JavaScript/DOM	Графический интерфейс менеджера модов
Nexus Engine	C++17	Ядро симуляции мира: NPC, экономика, карта, бои, караваны
meterea_mod_sdk.h	C API (DLL/SO)	Стабильный ABI для нативных плагинов с отложенными мутациями
ModKit.py	Python/PyQt6	Визуальный редактор модов (Creation Kit)
Песочница (Sandbox)	JS Proxy	Изоляция модов: проксирование window/document, ограничение доступа к Node.js и Electron

1.2. Жизненный цикл загрузки модов

Процесс загрузки модов следует строгой последовательности, гарантирующей корректную инициализацию всех компонентов:

Шаг 1. ModKit инициализация — функция `initModKit()` считывает настройки из конфигурации пользователя (список активных модов, режим песочницы) через IPC-канал Electron.

Шаг 2. Загрузка манифестов — для каждого активного мода считывается файл `mod.json`, проверяется корректность полей (`id`, `name`, `version`), валидируются зависимости.

Шаг 3. Загрузка скриптов — скрипты из поля `scripts` выполняются в песочнице (если включен безопасный режим) или напрямую через `AsyncFunction`. Коду передаются `ModAPI`, `modId`, `modMeta`.

Шаг 4. Загрузка данных — JSON-файлы из поля `data` загружаются и глубоко сливаются с ванильной базой данных через `mergeDeep` с поддержкой `$delete`, `$replace`, `$push`.

Шаг 5. Сборка базы данных — все игровые данные (предметы, рецепты, биомы, расы, монстры и т.д.) объединяются в единую структуру `database`.

Шаг 6. Инициализация C++ ядра — база данных отправляется в Nexus Engine через IPC, ядро создаёт мир и начинает симуляцию.

Шаг 7. Вызов хука `onModsInitialized` — моды получают финальный сигнал готовности, после чего могут регистрировать периодические обработчики.

2. Быстрый старт: создание первого мода

В этом разделе мы создадим простой мод, добавляющий новые предметы и локализацию, не требующий написания кода. Такой подход называется декларативным моддингом — всё, что нужно — это JSON-файлы и манифест.

2.1. Структура папки мода

Каждый мод располагается в отдельной папке внутри директории `mods`. Путь к директории модов зависит от операционной системы:

ОС	Путь
Windows	%APPDATA%\chronicles-of-metereia\mods\
macOS	~/Library/Application Support/chronicles-of-metereia/mods/
Linux	~/config/chronicles-of-metereia/mods/

Минимальная структура папки мода выглядит следующим образом:

```
my_first_mod/ mod.json # Обязательный манифест мода data/ # Директория для JSON-данных
items.json # Переопределение/добавление предметов locales/ # Директория для локализации
ru.json # Русские строки en.json # Английские строки
```

2.2. Файл mod.json — манифест мода

Файл mod.json является единственным обязательным файлом мода. Он содержит метаданные и указывает движку, какие данные и скрипты нужно загрузить. Вот минимальный пример:

```
{ "id": "my_first_mod", "name": "My First Mod", "version": "1.0.0", "author": "YourName", "description": "Мой первый мод для Метеры", "dependencies": ["base_game"], "data": { "items": ["data/items.json"] } }
```

Поле id должно содержать только строчные латинские буквы, цифры и символ подчёркивания, начинаться с буквы. Поле dependencies перечисляет моды, которые должны быть загружены до данного; base_game — это встроенный псевдомод, представляющий ядро игры. Поле data определяет, какие типы данных мод предоставляет и из каких файлов их читать.

2.3. Добавление предметов

Создайте файл data/items.json. Формат предметов основан на системе тегов: каждый предмет имеет уникальный ID, название (мультиязычное по эпохам), базовую цену, категорию, теги и опциональные свойства:

```
{ "magic_crystal": { "names": { "rebirth": "Магический кристалл", "architects": "Энергоядро", "sundering": "Осколок Бездны", "silence": "Микрочип" }, "basePrice": 75, "category": "magic_raw", "tags": ["magic_raw", "crystal", "luxury"], "shelfLife": 999999, "properties": { "weight": 0.3, "glow_intensity": 8.5 } } }
```

Объект names содержит локализованные названия для каждой игровой эпохи (rebirth, architects, sundering, silence). Теги используются движком для категоризации: тег "food" делает предмет едой, "weapon" — оружием, "currency" — валютой. Поле shelfLife указывает срок годности в днях (999999 = не портится). Свойства в properties могут быть любыми и доступны через систему custom_props в C++ ядре.

2.4. Установка и активация мода

Поместите папку мода в директорию mods, запустите игру и откройте Менеджер Модификаций через кнопку в главном меню. Ваш мод появится в списке доступных. Нажмите кнопку "+" рядом с ним для активации, затем используйте стрелки вверх/вниз для настройки порядка загрузки. Порядок важен: если два мода изменяют одни и те же данные, последний в списке перезапишет предыдущий. Мод base_game всегда загружается первым и не может быть перемещён или отключён.

3. Справочник манифеста mod.json

Файл mod.json — это центральный конфигурационный файл каждого мода. Он определяет метаданные, зависимости, скрипты и данные мода. В этой главе подробно описано каждое поле манифеста с примерами и ограничениями.

3.1. Поля манифеста

Поле	Тип	Обязательно	Описание
id	string	Да	Уникальный идентификатор: строчные буквы, цифры, _
name	string	Да	Человекочитаемое название мода
version	string	Да	Версия в формате SemVer (например, 1.0.0)
author	string	Нет	Имя автора мода
description	string	Нет	Описание мода (показывается в менеджере)
dependencies	string[]	Нет	Массив ID модов, которые должны быть загружены раньше
scripts	string[]	Нет	Массив путей к JS-скриптам (от корня папки мода)
data	object	Нет	Мэппинг типов данных на массивы путей к JSON-файлам
total_conversion	boolean	Нет	Если true, ванильная БД не загружается
native_plugins	string[]	Нет	Массив путей к DLL/SO плагинам

3.2. Поле data — типы данных

Поле data определяет, какие игровые данные мод предоставляет. Каждый ключ — это тип данных, значение — массив путей к JSON-файлам. Файлы загружаются последовательно и глубоко сливаются с существующей базой данных. Поддерживаются следующие типы данных:

Ключ	Формат файла	Описание
items	Object (item_id -> {...})	Предметы: название, цена, теги, свойства
facilities	Object	Здания и предприятия: название, теги, обслуживание
locations	Array/Object	Локации для генерации мира
lore	Array of paths	Текстовые файлы лора (.txt)
biomes	Array	Биомы для карты мира
monsters	Array	Эпические монстры
disasters	Array	Бедствия (землетрясения, эпидемии и т.д.)
races	Array	Расы (человек, эльф, дварф, орк и др.)
professions	Array	Профессии NPC
traits	Array	Черты характера NPC
npc_names	Object	Имена NPC по расам
faction_relations	Object	Отношения между фракциями
city_gen	Object	Правила генерации городов
world_config	Object	Конфигурация генерации мира (карта, реки, вулканы)

3.3. Total Conversion — полная конверсия

Если в манифесте указано "total_conversion": true, то при загрузке ванильная база данных предметов (economy_items.json), рецептов (economy_recipes.json) и предприятий (facility_names.json) не загружается. Мод обязан предоставить собственные данные для этих категорий. Это полезно для модов, полностью меняющих сеттинг игры — например, киберпанк или постапокалипсис. При этом данные о биомах, расах и других реестрах всё равно могут наследоваться из ванилы, если мод не предоставляет собственные файлы для этих категорий.

3.4. Валидация манифеста

При загрузке ModLoader проверяет следующие условия: id обязательно и соответствует шаблону `^[a-z0-9_]+$`; name и version обязательны; dependencies и scripts должны быть массивами; total_conversion должен быть логическим значением. При обнаружении ошибок мод помечается как повреждённый, в UI отображается сообщение об ошибке, и мод не загружается. Проверка зависимостей происходит на этапе загрузки: если зависимость не загружена до текущего мода, выводится предупреждение в менеджере модов.

4. Декларативный моддинг: JSON-данные

Декларативный моддинг — это самый доступный способ изменения игры, не требующий программирования. Вы создаёте JSON-файлы с новыми данными, а движок автоматически загружает и объединяет их с существующей базой. В этой главе подробно описан формат каждого типа данных.

4.1. Предметы (items)

Предметы — фундаментальная единица экономики игры. Каждый предмет описывается объектом с уникальным ключом (ID предмета). Предметы в ванильной игре используют систему тегов для категоризации, а также мультязычные названия по эпохам. Данные предметов из модов глубоко сливаются с ванильными через mergeDeep, что позволяет переопределять отдельные поля, не переписывая весь предмет целиком.

Поля предмета:

Поле	Тип	Описание
names	Object	Названия по эпохам: rebirth, architects, sundering, silence
basePrice	number	Базовая цена предмета в золоте/кредитах
category	string	Категория: raw_food, processed_food, raw_material, weapon, armor, tool, vehicle, luxury, document, magic_raw, consumable, currency, metal_ingot, cloth, building

tags	string[]	Теги для движка: food, weapon, currency, melee, ranged, armor, medical, cyberware, mount, vehicle, tool и др.
shelfLife	number	Срок годности в игровых днях (999999 = не портится)
properties	Object	Произвольные свойства: nutrition, weight, spoil_rate, isTransport, speedMultiplier и др.

Пример полного описания предмета:

```
"plasma_rifle": { "names": { "rebirth": "Плазменная винтовка", "silence": "Прототип X-7" }, "basePrice": 800, "category": "weapon", "tags": ["weapon", "ranged", "energy"], "shelfLife": 999999, "properties": { "damage": 45, "range": 300, "weight": 4.5, "ammo_type": "energy_cell" } }
```

Ванильные предметы (economy_items.json) содержат около 40 базовых позиций: от пшеницы и мяса до оружия, брони и транспортных средств. Моды могут добавлять новые предметы, переопределять существующие или удалять их с помощью механизма слияния.

Операторы слияния mergeDeer

При объединении данных модов с ванильной базой используется функция mergeDeer с тремя специальными операторами:

Оператор	Пример	Действие
\$delete	"field": "\$delete"	Удаляет указанное поле из целевого объекта
\$replace	"arr": { "\$replace": [1,2,3] }	Полностью заменяет массив/значение вместо слияния
\$push	"arr": { "\$push": [4, 5] }	Добавляет элементы в конец существующего массива

4.2. Биомы (biomes)

Биомы определяют типы тайлов на карте мира. Каждый биом описывает проходимость, стоимость перемещения, водный статус, цвет на карте и правила генерации через шум Перлина. Формат массива:

```
[ { "id": "crystal_caves", "name": "Кристалльные пещеры", "numeric_id": 20, "color_hex": "#9966ff", "movement_cost": 3, "is_water": false, "is_impassable": false, "tags": ["land", "underground", "crystal"], "gen_rules": { "min_elev": -0.3, "max_elev": 0.1, "min_temp": 0.0, "max_temp": 0.4, "min_moist": 0.7, "max_moist": 1.0 } } ]
```

Поле numeric_id должно быть уникальным и не пересекаться с ванильными (0–18). Правила gen_rules определяют, при каких значениях высоты, температуры и влажности этот биом генерируется. Тайлы с movement_cost = 9999 непроходимы для наземных юнитов. Теги используются для привязки монстров к биомам и других механик. Ванильная игра содержит 19 биомов: от океана и равнин до вулканов и аномалий.

4.3. Монстры (monsters)

Монстры — это эпические существа, появляющиеся в определённых биомах и способные превращать тайлы в другие биомы. Каждый монстр описывается объектом с полями:

Поле	Тип	Описание
id	string	Уникальный идентификатор монстра
name	string	Отображаемое название
base_hp	number	Базовое здоровье
base_attack	number	Базовая атака
base_defense	number	Базовая защита
spawn_biome_tag	string	Тег биома, в котором появляется монстр
corrupt_biome_to	string	ID биома, в который превращается тайл при появлении
loot_table_id	string	ID таблицы лута (определяет добычу при убийстве)

В ванильной игре присутствуют четыре эпических монстра: Древний Дракон (горы), Кракен Бездны (океан), Король-Лич (руины) и Лорд Ифритов (пустыня). Мод может добавлять новых монстров или переопределять существующих, включая в JSON-файл записи с совпадающими ID.

4.4. Бедствия (disasters)

Бедствия — случайные события, поражающие регионы мира. Каждое бедствие имеет радиус поражения, длительность, урон населению и зданиям, допустимые климатические зоны и может трансформировать биомы. Формат:

Поле	Тип	Описание
id	string	Уникальный идентификатор
name	string	Название бедствия
base_radius	number	Радиус поражения в тайлах
base_duration_days	number	Длительность в игровых днях
population_damage_percent	number	Процент урона населению
facility_damage	number	Урон зданиям (абсолютный)
allowed_climates	string[]	Допустимые климаты: any, temperate, tropical, dry, mountain, coast, ocean
floods_tiles	boolean	Затапливает ли тайлы
ruins_roads	boolean	Разрушает ли дороги
transform_biome_to	string	ID биома, в который превращается поражённый
affected_biomes	string[]	Список биомов, которые могут быть поражены
stability_penalty	number	Штраф к стабильности региона
threat_set	number	Устанавливает уровень угрозы региона

fertility_mult	number	Множитель плодородия (1.0 = нет изменения)
spawn_item	string	Предмет, появляющийся при бедствии
spawn_item_qty	number	Количество появляющегося предмета

4.5. Расы (races)

Расы определяют параметры персонажей и NPC. Каждая раса имеет модификаторы характеристик, предпочтения по фракциям и биомам, а также статистику по классам:

```
{ "id": "drow", "name": "Дроу", "base_race": false, "faction_preference":
["underdark_syndicate"], "biome_preference": "swamp", "stat_modifiers": { "str": 0, "
dex": 2, "int": 1, "con": 0, "cha": -1 }, "class_stats": { "warrior": { "str": 12, "dex":
14, "int": 8, "con": 10, "cha": 8, "res": 10 }, "mage": { "str": 8, "dex": 12, "int": 14, "
con": 9, "cha": 10, "res": 8 }, "default": { "str": 10, "dex": 12, "int": 10, "con": 10, "
cha": 8, "res": 10 } } }
```

Поле `base_race` указывает, является ли раса основной (человек в ваниле). Только одна раса может быть `base_race`. Статистика классов определяет начальные значения характеристик для NPC данной расы и класса. Если класс не указан, используется `default`.

4.6. Профессии (professions)

Профессии определяют род деятельности NPC. Каждая профессия связана с типом (`profession_type`), который используется движком для экономической симуляции, и опционально с инструментом (`tool_tag`) и шансом его наличия (`tool_chance`). Ванильные типы профессий: `farmer`, `artisan`, `merchant`, `innkeeper`, `ruler`, `cleric`, `mage`, `mercenary`, `fisherman`, `sailor`, `pirate`, `shipwright`, `gatherer`.

4.7. Черты характера (traits)

Черты характера — это особенности NPC, влияющие на их личность и поведение. Каждая черта задаёт смещение (`bias`) для параметров личности: `aggression`, `sociability`, `greed`, `loyalty`, `lust`. При генерации NPC черта применяется к базовым значениям личности, создавая уникальных персонажей.

4.8. Конфигурация мира (world_config)

Конфигурация мира определяет параметры генерации карты: размер, тип рельефа, параметры шума Перлина для континентов, настройки рек и вулканов. Это мощный инструмент для модов, меняющих масштаб и характер генерируемого мира.

```
{ "map_width": 256, "map_height": 256, "landform": "continent", "continent": { "
noise_frequency": 2.5, "noise_octaves": 6, "elevation_shift": 0.3, "edge_falloff_power":
2.0, "min_land_ratio": 0.5 }, "rivers": { "noise_frequency": 3.0, "noise_octaves": 3, "
threshold_default": 0.02 }, "volcanoes": { "count": 5, "min_radius": 3, "max_radius": 6 }
}
```

5. Скриптовый моддинг: ModAPI v3.0

ModAPI — это JavaScript API (версия 3.0), предоставляющее модам доступ к игровым системам. Объект `window.ModAPI` доступен глобально и содержит методы для регистрации хуков, команд, фильтров, виджетов и многого другого. Скрипты модов выполняются в песочнице (если включён безопасный режим) или в прямом контексте (если отключён).

5.1. Система хуков (on/emit)

Система хуков — основной механизм взаимодействия модов с игрой. Моды подписываются на именованные события, а ядро игры генерирует эти события в соответствующие моменты. Подписка выполняется через `ModAPI.on()`, генерация — через `ModAPI.emit()`. Хуки поддерживают асинхронные обработчики.

Основные хуки жизненного цикла:

Хук	Аргументы	Когда вызывается
<code>onModsInitialized</code>	(нет)	После загрузки всех модов; моды могут выполнять финальную инициализацию
<code>onDatabaseLoad</code>	(database)	При загрузке базы данных; моды могут модифицировать database перед отправкой в C++ ядро
<code>onBeforeDailyTick</code>	(нет)	Перед ежедневным тиком симуляции; моды могут проверять/изменять состояние
<code>onLoreLoad</code>	(hookData)	При загрузке лора; моды могут переопределить hookData.lore

```
// Подписка на хук загрузки базы данных ModAPI.on('onDatabaseLoad', async (db) => { const myBiomes = await ModAPI.readJson('my_mod', 'data/biomes.json'); if (myBiomes) db.biomes = myBiomes; console.log('[MyMod] Биомы заменены!'); });
```

5.2. Hook Chain System (hookFunction)

Hook Chain System — механизм перехвата функций с приоритетами, заменивший небезопасное monkey-patching. Каждый хук получает оригинальную функцию и аргументы, может модифицировать результат или вызвать оригинал. Хуки выполняются в порядке приоритета: меньшее значение = выполняется раньше (pre-hook), большее = позже (post-hook). Значение по умолчанию — 100.

```
// Перехват функции advanceTime с приоритетом 50 (pre-hook) ModAPI.hookFunction(window, 'advanceTime', 'my_mod', (originalFunc, pulses) => { // Выполняем оригинальную логику originalFunc(pulses); // Добавляем свою логику после if (pulses > 0) { myModState.tick(pulses); } }, 50);
```

Для обратной совместимости сохранены устаревшие методы `patchFunction/unpatchFunction`, которые делегируют к `hookFunction/unhookFunction`. При выгрузке мода (`unloadMod`) все его хуки автоматически снимаются.

5.3. Инъекции в промпт ИИ

Метод `addPromptInjection()` добавляет текст в системный промпт, отправляемый ИИ-рассказчику. Это мощный инструмент для изменения поведения ИИ: добавления правил сеттинга, изменения стиля повествования или введения новых механик. Текст инъекции добавляется без ограничений на количество или длину.

```
ModAPI.addPromptInjection(` [МОД: DARK FANTASY] Мир жесток и беспощаден. Описывай страдания, грязь и отчаяние. Магия опасна и стоит здоровья кастующего. Каждый бой может быть последним. `);
```

5.4. Кастомные ГМ-команды

Метод `addCommand()` регистрирует команду, которую ИИ-рассказчик может вызывать в ходе игры. Команда получает объект аргументов и возвращает строку обратной связи. Документация команды (третий аргумент) добавляется в промпт ИИ для информирования о доступности команды.

```
ModAPI.addCommand('cast_spell', async (args) => { const spellName = args.spell || 'Огненный шар'; const manaCost = args.cost || 10; if (player.stats.mp < manaCost) { return `Недостаточно маны для заклинания ${spellName}!`; } await ModAPI.sendToEngine('updateStat', { stat: 'mp', change: -manaCost }); const dmg = Math.floor(Math.random() * 30) + 20; return `Заклинание ${spellName} нанесло ${dmg} урона!`; }, { name: 'cast_spell', description: 'Применить заклинание (бросок на Интеллект).', args: '{ "spell": "Название", "cost": 10 }' });
```

5.5. Фильтры (prompt, response, text)

Фильтры позволяют перехватывать и модифицировать текст на различных этапах взаимодействия с ИИ. Это полезно для цензуры, автозамены терминов и адаптации ответов под сеттинг мода.

Метод	Когда вызывается	Сигнатура callback
<code>addPromptFilter(cb)</code>	Перед отправкой промпта ИИ	<code>(promptText) => modifiedText</code>
<code>addResponseFilter(cb)</code>	При получении ответа ИИ	<code>(responseText) => modifiedText</code>
<code>addTextFilter(cb)</code>	При отображении любого текста	<code>(text) => modifiedText</code>

```
// Автозамена "золото" на "кредиты" в ответах ИИ ModAPI.addResponseFilter((text) => { return text.replace(/золот(о|а|ые|ых)/gi, 'кредитов') .replace(/монет(ы|ам|ами)/gi, 'чипов'); });
```

5.6. Система виджетов (registerWidget)

Система виджетов позволяет добавлять UI-элементы в предопределённые зоны интерфейса. Это предпочтительный способ модификации UI вместо устаревшего `addUI()`. Виджеты привязываются к регионам (`regionId`) и автоматически рендерятся при обновлении интерфейса.

```
// Регистрация виджета с HTML и callback после рендера ModAPI.registerWidget( 'character-bottom', // regionId 'my_mod_toxicity', // widgetId (уникальный) `<div id="toxicity-bar" style="..."> <div style="width: 0%; background: #ff0;"></div> </div>`, // HTML-контент (element) => { // onRender callback updateBar(element); } ); // Альтернатива: render-функция для React/Vue ModAPI.registerWidget('inventory-top', '
```

```
react_widget', (container) => { ReactDOM.render(<MyComponent />, container); } );
```

Доступные регионы виджетов: character-bottom, character-top, inventory-bottom, inventory-top и другие, определяемые шаблонами интерфейса. HTML-контент виджетов проходит санитизацию: удаляются script-теги, опасные iframe/object/embed элементы, обработчики событий on* и javascript:-URL.

5.7. CSS-инъекции (addStyle)

Метод addStyle() позволяет добавлять произвольные CSS-стили для изменения визуального оформления игры. Стили автоматически привязываются к моду и удаляются при его выгрузке. Это основной инструмент для темизации интерфейса.

```
// Добавление стиля с ID для отслеживания ModAPI.addStyle('cyberpunk_theme', `:root {
--bg-base: #020204 !important; --accent-blue: #00f3ff !important; --accent-red: #ff003c !
important; } .bubble-gm { border-left: 4px solid var(--accent-blue) !important; } `); //
Удаление стиля по ID ModAPI.removeStyle('cyberpunk_theme');
```

5.8. Горячие клавиши (registerHotkey)

```
ModAPI.registerHotkey('Ctrl+Shift+M', () => { ModAPI.notify('Мод-меню открыто!');
toggleModMenu(); }); // Удаление хоткея ModAPI.unregisterHotkey('Ctrl+Shift+M');
```

5.9. Локализация (addTranslations/setString)

```
// Загрузка переводов из JSON-файлов мода const ruLoc = await ModAPI.readJson(modId, '
locales/ru.json'); if (ruLoc) ModAPI.addTranslations('ru', ruLoc); // Переопределение
конкретной строки ModAPI.setString('ru', 'gameInterface.characterPanel.gold', 'Кредиты');
```

5.10. Система сохранений (registerSaveData)

Метод registerSaveData() позволяет моду сохранять собственное состояние между сессиями. При сохранении игры вызывается callback onSave, возвращающий сериализуемые данные. При загрузке — callback onLoad, получающий сохранённые данные. Это критически важно для модов с состоянием (например, уровень токсичности, квестовые флаги).

```
let toxicity = 0; ModAPI.registerSaveData( 'my_mod', () => ({ toxicity, lastUpdate:
Date.now() })), // onSave (data) => { // onLoad if (data) toxicity = data.toxicity || 0; } )
;
```

5.11. Взаимодействие с C++ ядром

ModAPI предоставляет два метода для отправки команд в C++ ядро (Nexus Engine) через IPC:

Метод	Назначение	Пример вызова
sendToEngine(cmd, args)	Высокоуровневая ГМ-команда	ModAPI.sendToEngine("updateStat", {stat:"hp", change:-5})
sendRawToEngine(cmd, args)	Низкоуровневая команда ядра	ModAPI.sendRawToEngine("applyModChanges", {mutations: [...]})

Метод `queueMutation()` ставит мутацию в очередь для пакетной отправки в C++ ядро. Это снижает количество IPC-вызовов и повышает производительность. Мутации применяются на следующем тике симуляции (deferred response pattern). Метод возвращает Promise, который резолвится после подтверждения от ядра. Важно: поле `type` в объекте мутации должно точно совпадать со строкой, которую ожидает C++ парсер (`applyModChanges`). Например, для установки стабильности используйте `"setStability"`, а не `"setRegionStability"`; для изменения населения — `"modifyPopulation"`, а не `"modifyRegionPopulation"`. Несовпадение типа приведёт к тому, что мутация будет проигнорирована ядром.

```
// Пакетная отправка мутаций await ModAPI.queueMutation({ type: 'setStability',
region_id: 'region_001', value: 50 });
```

5.12. Чтение файлов мода

```
// Чтение текстового файла const content = await ModAPI.readFile('my_mod', '
data/lore.txt'); // Чтение JSON-файла (автопарсинг) const items = await
ModAPI.readJson('my_mod', 'data/items.json'); if (items) { console.log('Загружено
предметов:', Object.keys(items).length); }
```

5.13. Виртуальная файловая система (resolveAsset)

Метод `resolveAsset()` формирует URL для доступа к ресурсам мода (изображения, звуки и т.д.) через кастомный протокол `metera-mod://`. Используется в HTML-элементах `img`, `audio`, `video` и CSS `background-image`.

```
// Получение URL изображения из папки мода const iconUrl = ModAPI.resolveAsset('my_mod', '
assets/icons/sword.png'); // Результат: "metera-mod://my_mod/assets/icons/sword.png" //
Использование в HTML ModAPI.registerWidget('character-bottom', 'my_widget', `![Sword](${iconUrl})
```

5.14. Уведомления и вкладки настроек

```
// Отправка уведомления в игровой лог ModAPI.notify('Квест обновлён!', 'level-up'); //
Добавление вкладки в настройки ModAPI.addSettingsTab('my_mod_settings', 'Настройки мода',
` <div class="setting-item"> <label>Включить функцию X</label> <input type="checkbox"
id="feature-x-toggle" /> </div> `);
```

5.15. Выгрузка мода (unloadMod)

При выгрузке мода ModAPI автоматически выполняет полную очистку: удаляет инжецированные CSS-стили, снимает все хуки (`hookFunction`), удаляет виджеты, отменяет таймеры, снимает обработчики сохранений, удаляет кастомные команды и хоткеи, очищает CSS-классы темы. Это гарантирует, что выгрузка мода не оставляет "мусора" в игровом состоянии.

6. Тотальная конверсия (Total Conversion)

Тотальная конверсия — это тип мода, полностью заменяющий игровой сеттинг. При активации флага `total_conversion` ванильная база данных предметов, рецептов и предприятий не загружается, и мод обязан

предоставить собственные данные. Это позволяет превратить фэнтези-Метеру в киберпанк, постапокалипсис, космическую оперу или любой другой сеттинг.

6.1. Структура мода тотальной конверсии

Мод тотальной конверсии `cyberpunk_core` (входит в репозиторий как пример) демонстрирует все ключевые приёмы:

```
{ "id": "cyberpunk_core", "name": "Neon & Chrome: Total Conversion", "version": "1.0.0", "total_conversion": true, "dependencies": ["base_game"], "scripts": ["scripts/main.js"], "data": { "items": ["data/items.json", "data/items_expanded.json"], "facilities": ["data/facilities.json"], "locations": ["data/locations.json", "data/locations_expanded.json"], "lore": ["data/lore.txt", "data/lore_expanded.txt"] } }
```

6.2. Ключевые техники тотальной конверсии

1. Подмена базы данных через хук `onDatabaseLoad`. Скрипт мода подписывается на хук `onDatabaseLoad` и заменяет все ключевые реестры базы данных: биомы, монстры, расы, профессии, черты, имена NPC, отношения фракций, генерацию городов. Это позволяет полностью изменить правила генерации мира.
2. Инъекция в промпт ИИ. Мод добавляет масштабную инъекцию в системный промпт, указывая ИИ следовать новому сеттингу. Это критически важно для тотальной конверсии, так как ИИ по умолчанию ориентируется на фэнтези-сеттинг Метеры. Инъекция должна содержать чёткие инструкции о замене понятий: магия = скрипты, оружие = огнестрел, золото = кредиты и т.д.
3. CSS-тематизация. Мод добавляет CSS, полностью меняющий визуальное оформление: цветовую схему, шрифты, эффекты (CRT-сканлайны, неоновое свечение), стили панелей и кнопок. Это создаёт визуальную целостность нового сеттинга.
4. Кастомные механики через ГМ-команды. Мод регистрирует команды для уникальных механик: `netrun_hack` (взлом систем), `install_cyberware` (установка имплантов), `firearm_shoot` (огнестрельный бой). Каждая команда реализует броски кубиков и взаимодействие с C++ ядром через `sendToEngine`.
5. Периодические события. Мод подписывается на хук `onBeforeDailyTick` для реализации механик, срабатывающих каждый игровой день: проверка киберпсихоза, начисление токсичности и т.д.

7. Нативный C/C++ моддинг (Metera Mod SDK)

Metera Mod SDK (`meterea_mod_sdk.h`) — это стабильный C-API для создания нативных плагинов (DLL/SO), работающих непосредственно внутри C++ ядра симуляции (Nexus Engine). API использует непрозрачные дескрипторы (opaque handles) и строго типизированные extern "C" функции, гарантируя стабильность ABI между версиями движка.

7.1. Жизненный цикл нативного плагина

Нативный плагин проходит следующие этапы:

1. Загрузка DLL/SO. Движок загружает библиотеку через `dlopen` (Linux/macOS) или `LoadLibrary` (Windows).
2. `MeteraPlugin_GetAPI()`. Движок вызывает эту функцию, передавая указатель на таблицу API (`MeteraAPI`). Плагин сохраняет этот указатель для последующего использования.
3. `MeteraPlugin_Init(plugin_id)`. Движок назначает плагину уникальный ID. Плагин регистрирует свои callback-функции для хуков (ежедневный тик, часовой тик, смена региона, смерть NPC, битва, торговля, бедствие, постройка здания).
4. `MeteraPlugin_OnLoad()`. Опциональный вызов для одноразовой инициализации (загрузка конфигурации, выделение ресурсов).
5. Выполнение. В ходе симуляции движок вызывает зарегистрированные callback-функции. Мутации мира откладываются до следующего тика (`deferred response`).
6. `MeteraPlugin_Shutdown()`. При завершении работы плагин освобождает все ресурсы.

7.2. Обязательные экспортируемые функции

Каждый нативный плагин ДОЛЖЕН реализовать следующие функции с C-связью (`extern "C"`):

Функция	Возвращаемый тип	Назначение
<code>MeteraPlugin_GetName()</code>	<code>const char*</code>	Возвращает статическое имя плагина
<code>MeteraPlugin_GetVersion()</code>	<code>const char*</code>	Возвращает строку версии плагина
<code>MeteraPlugin_GetAPI(const MeteraAPI* api)</code>	<code>void</code>	Получает таблицу API от движка
<code>MeteraPlugin_Init(int32_t plugin_id)</code>	<code>MeteraResult</code>	Инициализация; возвращает <code>METERA_OK</code> при успехе
<code>MeteraPlugin_OnLoad()</code>	<code>void</code>	Опциональная одноразовая настройка
<code>MeteraPlugin_Shutdown()</code>	<code>void</code>	Опциональная очистка при завершении

7.3. Таблица API (MeteraAPI)

Структура `MeteraAPI` передаётся плагину через `MeteraPlugin_GetAPI()` и содержит указатели на все функции, предоставляемые движком. API разделено на три категории: запросы мира (только чтение), мутации мира (отложенные) и утилиты.

7.3.1. Запросы мира (read-only)

Эти функции возвращают текущее состояние мира без возможности его изменения:

Функция	Возвращаемый тип	Описание
getRegionPopulation(region_id)	int32_t	Население региона (-1 при ошибке)
getRegionStability(region_id)	int32_t	Стабильность региона (0-100)
getRegionFaction(region_id)	const char*	ID фракции, контролирующей регион
getRegionBiome(region_id)	const char*	ID биома региона
getWorldPopulation()	int64_t	Общее население мира
getCurrentDay()	int32_t	Текущий игровой день
getCurrentHour()	int32_t	Текущий час (0-23)
getRegionNpcCount(region_id)	int32_t	Количество NPC в регионе
getItemPrice(item_id)	double	Цена предмета из базы экономики
getGlobalString(key)	const char*	Глобальная строка мода (для обмена данными)
getNpcHp(npc_id)	int32_t	Текущее HP NPC
getContainerItemCount(container_id, item_prototype)	int32_t	Количество предметов в контейнере
getMapWidth() / getMapHeight()	int32_t	Размеры карты в тайлах
getTileBiome(x, y)	int32_t	ID биома тайла (-1 при выходе за границы)
getFactionRelation(f1, f2)	int32_t	Отношение между двумя фракциями
getFactionState(faction_id)	const char*	Состояние фракции
getRegionMoneySupply(region_id)	double	Денежная масса в регионе
getBusinessCash(business_id)	int32_t	Касса предприятия
getNpcLocation(npc_id)	const char*	Текущая локация NPC
getNpcGold(npc_id)	int32_t	Золото NPC
getRegionThreat(region_id)	int32_t	Уровень угрозы региона

7.3.2. Мутации мира (deferred)

Эти функции изменяют состояние мира. Изменения применяются на следующем тике симуляции (deferred response pattern), что предотвращает блокировку цикла симуляции. Все функции возвращают MeteraResult (METERA_OK при успехе):

Функция	Описание
setStability(region_id, value)	Установить стабильность региона (0-100). JSON-тип мутации: "setStability"
modifyPopulation(region_id, delta)	Изменить население на delta. JSON-тип мутации: "modifyPopulation"

multiplyAllPrices(factor)	Умножить все цены на коэффициент (для конверсий)
multiplyItemPrice(item_id, factor)	Умножить цену конкретного предмета
setGlobalString(key, value)	Установить глобальную строку (обмен данными между модами)
spawnItem(item_id, quantity, container_id)	Создать предмет в контейнере
triggerDisaster(type, region_id, severity)	Вызвать бедствие в регионе
spawnMonster(type, region_id)	Породить эпического монстра в регионе
addNews(text, location, importance, category)	Добавить новость в хроники мира
setNpcHp(npc_id, hp)	Установить HP NPC (применяется немедленно для боевой логики)
setTileBiome(x, y, biome_id)	Изменить биом тайла
setFactionRelation(f1, f2, value)	Установить отношение между фракциями
forceWar(f1, f2)	Объявить войну между фракциями
forcePeace(f1, f2)	Заклучить мир между фракциями
modifyRegionMoney(region_id, delta)	Изменить денежную массу региона
modifyBusinessCash(business_id, delta)	Изменить кассу предприятия
teleportNpc(npc_id, region_id)	Телепортировать NPC в другой регион
modifyNpcGold(npc_id, delta)	Изменить золото NPC
spawnArmy(faction_id, region_id, size)	Создать армию фракции в регионе
setRegionThreat(region_id, value)	Установить уровень угрозы региона
setRoadState(from, to, state)	Изменить состояние дороги между регионами

7.4. Система callback-функций

Плагин может подписаться на события симуляции, зарегистрировав callback-функции. Callback-функции выполняются ПОСЛЕ текущего тика. Доступные типы хуков (MeteraHookType):

Хук	Сигнатура callback	Описание
METERA_HOOK_ON_DAILY_TICK	void(int32_t day)	Ежедневный тик симуляции
METERA_HOOK_ON_HOURLY_TICK	void(int32_t day, int32_t hour)	Часовой тик
METERA_HOOK_ON_REGION_CHANGED	void(const char* region_id, const char* change_type)	Изменение состояния региона
METERA_HOOK_ON_NPC_DEATH	void(const char* npc_id, const char* cause)	Смерть NPC

METERA_HOOK_ON_BATTLE	void(const char* region_id, int32_t atk, int32_t def)	Битва в регионе
METERA_HOOK_ON_TRADE	void(const char* from, const char* to, const char* item, int32_t qty)	Торговля между регионами
METERA_HOOK_ON_DISASTER	void(const char* id, const char* region, int32_t severity)	Бедствие
METERA_HOOK_ON_BUILDING_BUILT	void(const char* region_id, const char* type)	Постройка здания

7.5. Пример нативного плагина

```
// my_plugin.cpp #include "meterea_mod_sdk.h" #include <stdio.h> static const MeteraAPI* g_api = nullptr; static int32_t g_plugin_id = -1; static void onDailyTick(int32_t day) { g_api->log("MyPlugin: daily tick"); int32_t pop = g_api->getWorldPopulation(); if (pop > 100000 && day % 30 == 0) { g_api->triggerDisaster("plague", "region_capital", 3); } } extern "C" { METERA_EXPORT const char* MeteraPlugin_GetName() { return "MyPlugin"; } METERA_EXPORT const char* MeteraPlugin_GetVersion() { return "1.0.0"; } METERA_EXPORT void MeteraPlugin_GetAPI(const MeteraAPI* api) { g_api = api; } METERA_EXPORT MeteraResult MeteraPlugin_Init(int32_t pid) { g_plugin_id = pid; g_api->log("MyPlugin initialized"); return METERA_OK; } METERA_EXPORT void MeteraPlugin_OnLoad() { g_api->log("MyPlugin loaded"); } METERA_EXPORT void MeteraPlugin_Shutdown() { g_api->log("MyPlugin shutdown"); } }
```

Компиляция (Linux): g++ -shared -fPIC -o my_plugin.so my_plugin.cpp -I./engine. Компиляция (Windows): cl /LD my_plugin.cpp /I./engine /Fe:my_plugin.dll. Путь к скомпилированному плагину указывается в поле native_plugins манифеста mod.json. При активации мода с нативным плагином пользователю показывается предупреждение о безопасности.

7.6. Безопасность нативных плагинов

Нативные плагины выполняются с полными привилегиями процесса и могут получить доступ к файловой системе, сети и другим ресурсам. Поэтому в менеджере модов предусмотрен Безопасный режим (Safe Mode): при его активации нативные плагины блокируются и не загружаются. При попытке включить мод с нативным плагином пользователь получает предупреждение с подтверждением. Менеджер UI помечает такие моды бейджем "NATIVE DLL". Рекомендуется включать Safe Mode при использовании модов из ненадёжных источников.

8. Система безопасности: Песочница (Sandbox)

Система песочницы обеспечивает частичную изоляцию кода модов от системных глобальных объектов. При включённом безопасном режиме (Safe Mode в настройках) скрипты модов выполняются в специально созданном окружении через AsyncFunction с параметризованными аргументами. Доступ к

Node.js (require, module, process) и Electron (electronAPI) блокируется, однако сетевые API (fetch, XMLHttpRequest, WebSocket, EventSource) и конструктор Function намеренно доступны модам — это осознанное проектное решение, позволяющее модам взаимодействовать с внешними сервисами.

8.1. Принцип работы

В безопасном режиме код мода выполняется через `new AsyncFunction(...paramNames, code).call(null, ..paramValues)`. Мод получает только те идентификаторы, которые явно переданы как параметры функции: `ModAPI` (замороженный фасад), `modId`, `modMeta`. Объекты `require`, `module`, `process` и `electronAPI` передаются как `undefined`, затеняя реальные глобальные переменные. Однако сетевые API (`fetch`, `XMLHttpRequest`, `WebSocket`, `EventSource`) и конструктор `Function` намеренно доступны модам — они передаются напрямую в песочницу через `safeGlobals`, что позволяет модам взаимодействовать с внешними сервисами. Конструктор `eval` намеренно не передаётся как параметр, но остаётся доступен через глобальный контекст.

8.2. Проксирование window и document

Доступ к `window` осуществляется через безопасный прокси (`createSafeWindowProxy`), который пропускает игровые глобальные переменные (`player`, `t`, `updateCharacterSheet`) и ограничивает доступ к системным свойствам. Важно: сетевые API и конструктор `Function` намеренно НЕ блокируются — они доступны модам для легитимного использования:

Свойство	Статус	Примечание
fetch, XMLHttpRequest, WebSocket, EventSource	Разрешено	Передаются напрямую через safeGlobals для работы модов с сетью
Function	Разрешено	Доступен для динамического создания функций
eval	Доступно косвенно	Не передаётся как параметр, но доступен через глобальный контекст
electronAPI	Блокировано	Доступ к IPC должен идти через ModAPI
require, module, process	Блокировано	Доступ к Node.js запрещён
localStorage, sessionStorage	Блокировано	Сохранения через registerSaveData
importScripts, Navigator	Блокировано	Загрузка внешних скриптов запрещена
constructor, __proto__, prototype	Блокировано	Предотвращение prototype pollution
open, close, postMessage	Блокировано	Предотвращение навигации и обмена сообщениями

Доступ к `document` осуществляется через прокси (`createDocumentProxy`), который блокирует создание опасных элементов (`script`, `iframe`, `object`, `embed`, `link`, `base`, `meta`, `form`), перехватывает `innerHTML/outerHTML` для удаления `script`-тегов и блокирует установку обработчиков событий `on*`.

8.3. Публичный фасад ModAPI

В песочнице модам доступен не оригинальный объект ModAPI, а его замороженный фасад (safeModAPIFacade), содержащий только публичные методы, привязанные через bind. Моды не могут напрямую обращаться к внутренним полям (_hookChains, hooks, mods, _pendingMutations и др.). Это предотвращает случайное или злонамеренное изменение внутреннего состояния API.

8.4. Отключение песочницы

Если безопасный режим отключён (Safe Mode = false в настройках), код мода выполняется напрямую через AsyncFunction без проксирования. Это полезно для легитимных модов, которым нужен доступ к полному глобальному контексту, но снижает безопасность. Разработчики модов должны тестировать свои моды как в безопасном, так и в небезопасном режиме.

9. Meterea Creation Kit (ModKit.py)

Meterea Creation Kit — это настольное приложение на PyQt6, предоставляющее визуальный интерфейс для создания и редактирования модов. Оно разработано для упрощения процесса моддинга, особенно для пользователей, не привыкших к ручному редактированию JSON.

9.1. Возможности Creation Kit

Менеджер модов: Левая панель отображает список установленных модов. При выборе мода в средней панели показывается дерево его файлов с возможностью создания новых файлов и папок. Кнопка "Настройки мода" открывает mod.json для редактирования.

Текстовый редактор: Правая панель содержит полнофункциональный текстовый редактор с подсветкой синтаксиса JSON (ключи, строки, числа, булевы значения, комментарии). Шрифт Consolas 11pt обеспечивает удобное чтение кода.

Визуальный редактор JSON: Для JSON-файлов доступен переключатель между текстовым и визуальным режимами. Визуальный режим отображает JSON в виде дерева (QTreeWidget) с возможностью добавления, удаления и изменения типа элементов через контекстное меню.

Валидатор JSON: Кнопка "Проверить JSON" выполняет синтаксическую проверку текущего файла и выводит результат. Это помогает обнаружить ошибки до запуска игры.

Создание мода: Кнопка "Новый Мод" создаёт папку мода с базовой структурой: mod.json, data/, main.js. Идентификатор мода автоматически нормализуется (строчные буквы, замена пробелов на подчёркивания).

Упаковка в ZIP: Кнопка "Упаковать в ZIP" создаёт архив мода для распространения. Архив содержит все файлы из папки мода.

Тёмная тема: Creation Kit использует тёмную тему в стиле VS Code, снижающую нагрузку на глаза при длительной работе.

9.2. Запуск Creation Kit

```
python3 ModKit.py
```

Требования: Python 3.9+, PyQt6. Директория модов определяется автоматически по операционной системе.

10. Менеджер модификаций (UI)

Менеджер модификаций — это встроенный интерфейс игры для управления модами. Он доступен из главного меню через кнопку "Mods" и предоставляет полный контроль над списком активных модов, порядком их загрузки и настройками безопасности.

10.1. Интерфейс менеджера

Список активных модов (левая панель) отображает моды в порядке загрузки. Мод `base_game` (Core/Ядро Игры) всегда первый и не может быть перемещён или отключён. Для каждого мода показаны название, версия, автор и бейдж "NATIVE DLL" при наличии нативных плагинов.

Список доступных модов (правая панель) содержит моды, установленные в папку `mods`, но не активированные. Нажатие "+" добавляет мод в конец списка активных.

Панель деталей отображает полную информацию о выбранном моде: название, версия, автор, описание и зависимости. Если файл `mod.json` повреждён, показывается сообщение об ошибке.

Управление порядком загрузки: Стрелки вверх/вниз изменяют приоритет мода. Моды загружаются сверху вниз; при конфликте данных побеждает последний в списке. Зависимости должны быть загружены до мода, который их требует; валидатор автоматически проверяет это условие и показывает ошибки.

10.2. Безопасный режим

Чекбокс "Безопасный режим" в верхней части менеджера управляет блокировкой нативных плагинов. При активации все DLL/SO плагины блокируются при загрузке, что защищает от потенциально вредоносного кода. Настройка сохраняется в конфигурации пользователя и применяется при следующем запуске.

11. Примеры модов из репозитория

Репозиторий содержит два примера модов, демонстрирующих различные подходы к моддингу:

11.1. cyberpunk_core — Neon & Chrome: Total Conversion

Полная конверсия игры в сеттинг киберпанка. Реализует: замену базы данных (предметы, биомы, расы, монстры), CSS-тематизацию в стиле CRT/неон, кастомные ГМ-команды (netrun_hack, install_cyberware, firearm_shoot), механику киберпсихоза (onBeforeDailyTick), инъекцию в промпт ИИ, мультиязычную локализацию. Флаг total_conversion: true запрещает загрузку ванильной БД.

11.2. neon_abyss — Neon Abyss

Мод-оверлей, превращающий Метеру в подземный киберпанк без полной конверсии. Реализует: инъекцию в промпт ИИ для смены атмосферы, кастомную команду hackTerminal, CSS-тематизацию, систему токсичности с виджетом и сохранениями, перехват функции advanceTime через patchFunction, фильтр ответов ИИ для автозамены "золото" на "кредиты". Не использует total_conversion, поэтому ванильная БД сохраняется.

Приложение А: Полный справочник ModAPI

Метод	Параметры	Возвращает	Описание
on	eventName, callback	void	Подписка на событие
emit	eventName, ...args	Promise	Генерация события
unregisterHook	eventName, callback	void	Отписка от события
hookFunction	obj, funcName, modId, callback, priority?	void	Перехват функции с приоритетом
unhookFunction	obj, funcName, modId	void	Снятие перехвата
addCommand	name, handler, docs?	void	Регистрация ГМ-команды
removeCommand	name	void	Удаление ГМ-команды
addPromptInjection	text	void	Инъекция в промпт ИИ
addPromptFilter	callback	void	Фильтр промптов
addResponseFilter	callback	void	Фильтр ответов ИИ
addTextFilter	callback	void	Глобальный текстовый фильтр
applyTextFilters	text	string	Применить все текстовые фильтры
registerWidget	regionId, widgetId, content, onRender?	void	Регистрация UI-виджета

renderWidgets	regionId, container	void	Рендер виджетов региона
addStyle	idOrCss, cssString?	void	Добавить CSS-стиль
removeStyle	id	void	Удалить CSS-стиль
registerHotkey	keyCombo, callback	void	Зарегистрировать хоткей
unregisterHotkey	keyCombo	void	Удалить хоткей
addTranslations	lang, translationsObj	void	Добавить переводы
setString	lang, path, value	void	Переопределить строку локализации
registerSaveData	modId, onSave, onLoad	void	Зарегистрировать обработчик сохранений
removeSaveHandler	modId	void	Удалить обработчик сохранений
sendToEngine	command, args	Promise	Высокоуровневая команда ядру
sendRawToEngine	command, args	Promise	Низкоуровневая команда ядру
queueMutation	mutation	Promise	Поставить мутацию в очередь
applyModChanges	mutations[]	Promise	Применить массив мутаций
notify	message, type?	void	Уведомление в игровой лог
addSettingsTab	tabId, title, html	void	Добавить вкладку настроек
readFile	modId, fileName	Promise(string?)	Прочитать файл мода
readJson	modId, fileName	Promise(object?)	Прочитать JSON-файл мода
resolveAsset	modId, assetPath	string	URL ресурса через metera-mod://
mergeDeep	target, ...sources	object	Глубокое слияние объектов
unloadMod	modId	void	Полная выгрузка мода

Приложение В: Коды ошибок MeteraResult

Код	Имя	Описание
0	METERA_OK	Успешное выполнение операции
-1	METERA_ERR_INVALID_HANDLE	Передан недействительный дескриптор
-2	METERA_ERR_NOT_FOUND	Запрашиваемый объект не найден
-3	METERA_ERR_INVALID_ARG	Недопустимый аргумент функции
-4	METERA_ERR_PERMISSION	Недостаточно прав для операции
-5	METERA_ERR_OVERFLOW	Переполнение при выполнении операции